

ArtBridge 구현 현황표 (코드 기준 v0.8)

정직성 원칙: Mock/부분구현은 사실대로 표기.

ArtBridge 기능별 구현현황 (코드 근거 기반, 정직 분류)

검증 근거: `src/lib/*` 서비스 계층, `src/app/api/*/route.ts`, `src/app/(app)/*` 서버액션, `src/auth.ts`, `README.md`. 모든 핵심 서비스는 Prisma(PostgreSQL)로 실제 DB에 기록한다. "Mock폴백"은 외부 의존성(PG·OpenAI·소셜) 한정이며, 비즈니스 로직·DB는 실동작이다.

기능	UI	API	DB	상태	비고
회원가입 (이메일/비번)	O	O	O	구현완료	<code>signup/actions.ts</code> + <code>bcryptjs(password.ts</code> , SALT 10) 해시. User 생성 실DB
로그인 (이메일/비번)	O	O	O	구현완료	Authjs v5 Credentials, JWT 세션, <code>verify Password</code> bcrypt 비교
Google 소셜로그인	O	O	O(Account)	조건부 활성화	버튼 항상 노출. <code>GOOGLE_CLIENT_ID/SECRET</code> 없으면 <code>loginWithGoogle</code> 이 <code>/login?error=google_unconfigured</code> 로 리다이렉트(크래시 방지). 키 있을 때만 실제 OAuth 작동
작가 탐색/목록	O	O	O	구현완료	<code>(app)/creators</code> 목록 + 북마크(<code>book marks.ts</code> , <code>@@unique</code> 토글). 실DB 조회
작가 상세 소개탭(포트폴리오)	O	-	O	구현완료	<code>StudioPortfolioIntro</code> + <code>StudioTabs</code> (<code>intro/posts/membership/artworks/club/community</code>), <code>?tab=</code> 쿼리 초기탭. CreatorProfile 확장필드 사용

기능	UI	API	DB	상태	비고
멤버십 가입	O	O	O	부분구현	<code>joinMembership</code> 이 <code>mockPaymentProvider</code> 를 직접 호출 — PG 키가 있어도 멤버십 결제는 항상 Mock(즉시 성공). Membership/Payment 실DB 생성
멤버십 관리(생성/취소)	O	O	O	구현완료	<code>membership-plans</code> POST 실DB, <code>cancelMembership</code> (소유자 검증: CANCELLED)
프로그램 등록/수정/삭제	O	O	O	구현완료	<code>programs.ts</code> 소유권 게이트 (@MX:ANCHOR), 실DB CRUD
프로그램 신청	O	O	O	구현완료	<code>applications.ts</code> , <code>programs/[id]/applications</code> 라우트
계약/양측서명/에스크로	O	O	O	구현완료	<code>contracts.ts</code> : 금액조율·팬/크리에이터 서명·트랜잭션. 에스크로 RELEASED는 <code>reviews.ts approveCompletion</code> 에서 처리
결제(PG)	O	O	O	Mock폴백	<code>resolvePaymentProvider()</code> env 분기: PG키 전부 있으면 <code>SandboxPaymentProvider</code> (PortOne/Toss, 단건조회 검증), 하나라도 없으면 <code>MockPaymentProvider</code> (외부호출 O, 항상 PAID). 계약·작품주문만 PG 분기 / 포스트구매·멤버십은 Mock 고정
정산/지급설정	O	O	O	구현완료	<code>payout-settings.ts</code> upsert(계좌 마스크·last4), <code>creator/settlements</code> GET. Settlement 레코드 생성/RELEASED 실DB
작품 커머스/주문	O	O	O	Mock폴백	<code>artwork-orders.ts</code> <code>purchaseArtwork</code> : 재고 트랜잭션·Payment·Settlement. 결제 provider는 <code>resolvePaymentProvider()</code> (키 없으면 Mock 즉시 PAID)
작품 배송/환불/이슈	O	O	O	구현완료	<code>artwork-fulfillment.ts</code> : shipment/refund/issues/resolve 라우트, 실DB 상태전이
포스트(공개/멤버/유료)	O	O	O	구현완료	<code>posts</code> CRUD + <code>post-access.ts</code> <code>canViewPost</code> (PUBLIC/MEMBER_ONLY/PAID 서버측 게이트)
포스트 유료구매	O	O	O	Mock 고정	<code>post-purchase.ts</code> 가 <code>mockPaymentProvider</code> 직접 호출 — PG키 무관 항상 Mock. Payment/Settlement 실DB
커뮤니티	O	O	O	구현완료	<code>community-access.ts</code> <code>canAccessCommunity</code> (멤버/결제참여/소유자), <code>community-posts</code> CRUD 실DB

기능	UI	API	DB	상태	비고
알림	O	O	O	구현완료	<code>notifications.ts</code> 읽음/전체읽음, 각 서비스 트랜잭션 내 Notification 생성. 외부 푸시 없음(인앱 전용)
상호 리뷰	O	O	O	구현완료	<code>reviews.ts createReview</code> 양방향(팬 ↔크리에이터), @@unique 1회 강제, 완료승인 자격검증
AI 추천(프로그램/멤버십)	O	O	-	Mock폴백	<code>ai/suggest.ts</code> : <code>OPENAI_API_KEY</code> 있으면 OpenAI(gpt-4o-mini, JSON Schema), 없거나 실패/스키마위반 시 결정론적 Mock. 응답에 <code>source:"openai" "mock"</code> 표기
이미지 업로드	O	O	-	부분구현	<code>creator/uploads</code> 가 로컬 FS(<code>public/uploads/</code>)에 저장. 외부 스토리지 (Cloudinary/Supabase) 미연동 — README 는 "키 없으면 비활성화"라 명시하나 코드는 키 무관 로컬 저장 동작. 배포(서버리스/Vercel) 환경에선 영속 불가

폴백 정책 요약 (과장 금지 핵심)

- 결제: `resolvePaymentProvider()` 는 계약·작품주문에서만 사용. 멤버십 가입·포스트 유료구매는 `mockPaymentProvider` 를 코드상 직접 호출하므로 PG 키가 있어도 항상 Mock 결제. 실제 PG는 계약/작품주문 경로 + env 3종(STORE_ID/CHANNEL_KEY/API_SECRET) 모두 있을 때만.
- AI: 키 없거나 호출 실패 시 `suggestMock` (djb2 시드 결정론적). 예외를 throw하지 않음.
- 이미지: 외부 스토리지 미구현. 로컬 파일시스템 저장만 존재(데모 한정, 영속성 한계).
- 소셜로그인: env 없으면 OAuth 미실행·안내 리다이렉트.